gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843
Property-based Testing

Dr. Francisco Gomes de Oliveira Neto (he/him)

Hedy Lamarr

- Inventor and Actress
- Developed a radio guidance system that used spread spectrum and frequency hopping (similar to Bluetooth)

What we will learn today?

- What is **Property-based Testing?** What are properties in a SUT?
- Why do PBT tools minimise / shrink inputs?
- How do testers use PBT? What types of **test data** can be **generated**?
- What are the **patterns of properties** that we can create?
- **Why** should we use PBT? What are its **trade-off**?

© Francisco Oliveira Neto - 2022

Part 1:
Properties and Property-based Testing

© Francisco Oliveira Neto - 2022

What is Property-based Testing?

- PBT is a technique where we define a **set of properties** that the prorgam should adhere, and try to **find counterexamples** that causes the program to break those properties.

- Properties should be **generic and compliant to any input** of the program that meet specific conditions.

© Francisco Oliveira Neto - 2022

What is Property-based Testing?

Since we define properties…

we can **generate an immense number of "random"** test data to **verify** the properties.

© Francisco Oliveira Neto - 2022

What is PBT?

- For instance, if we are testing a **sum function** that adds two integers some examples of property are (should hold for **any integer x, y):**
  - `For all x, y; add(x, y) == add(y, x)`
  - `For all x, 0; add(x, 0) == x`

- **Implicit, or Derived oracles** are very helpful to defining properties.
- **Avoid thinking of specific input** values, and focus on sets of values

© Francisco Oliveira Neto - 2022

PBT – Exercise 1 - Distance

- The program below calculates the distance between two integers. Write at least 2-3 properties to apply PBT to the SUT below.

```
public static double dist(int x1, int x2) {
    return Math.abs(x1 - x2);
}
```


© Francisco Oliveira Neto - 2022

PBT – Exercise 1 - Distance

- Think of mathematical properties that should adhere to any integer values when calculating distances. For instance:

```
public static double dist(int x1, int x2) {
    return Math.abs(x1 - x2);
}
```

- `For all x1, x2: dist(x1,x2) == dist(x2,x1)`
- `For all x1: dist(x1,x1) == 0`
- `For all x1, x2: dist(x1,x2) >= 0`

© Francisco Oliveira Neto - 2022

PBT – Exercise 2 - Strings

- Write properties for testing a function that **reverses a String**. The function receives a String as input and returns the same string but reversed. Null Strings should return Exceptions, whereas empty strings should return empty strings.

```
public static String reverse(String word) { … }
```


© Francisco Oliveira Neto - 2022

Summary – Part 1

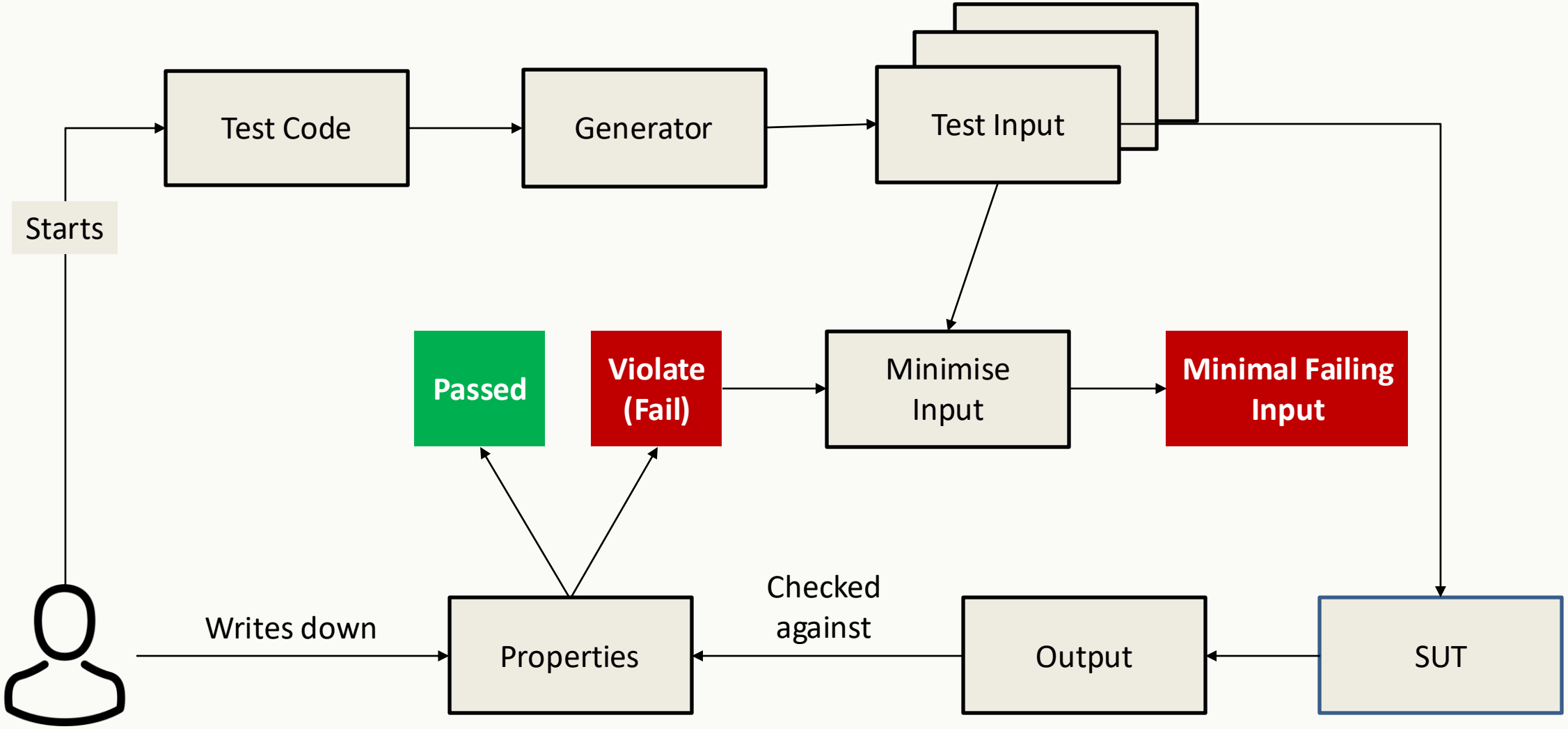
- Properties are defined **in terms of input space**.
- Should reflect **conditions** that the SUT must adhere.
- Properties should **not** be **specific** to a test input, rather, they should cover a set of test input.
- Defining properties **require creativity**. Implicit and Deribed oracles are very useful when devising properties.

© Francisco Oliveira Neto - 2022

Part 2:
Property-based Testing Frameworks and Automation

© Francisco Oliveira Neto - 2022

How does PBT works? (Workflow)



© Francisco Oliveira Neto - 2022

How does PBT works?

- PBT automation **frameworks rely** on two key features:
  1. Creation and **verification of properties**.
  2. Automatic **generation** of test data.

- Generators should provide a uniform distribution of possible cases. But some cases might require **custom distributions**.

© Francisco Oliveira Neto - 2022

How does PBT works?

- Generators for basic **data types** (integer, chars, etc.) or data structures (arrays, strings, etc.) are quite mature. The generation can be **customised** with filters to sample from a larger input space. For instance, **Jqwik** for Java.
  - *https://jqwik.net/docs/current/user-guide.html#how-to-use*

```
@Provide
private Arbitrary<String> randomHashtag() {
    return Arbitraries.strings().alpha().ofMinLength(2).ofMaxLength(100).
            withChars('#').filter(x -> x.startsWith("#"));
}
```


© Francisco Oliveira Neto - 2022

How does PBT works?

- Generators for ba
  strings, etc.) are
  sample from a la
  - https://jqwik.net

With **jqwik**, we can create **arbitrary strings** with **alphanumberic** characters that have length between **2 and 100 chars** that **starts with #**

```
@Provide
private Arbitrary<String> randomHashtag()
    return Arbitraries.strings().alpha().ofMinLength(2).ofMaxLength(100).
            withChars('#').filter(x -> x.startsWith("#"));
}
```


© Francisco Oliveira Neto - 2022

PBT – Minimising (Shrinking) cases

- Often there are **multiple test input that violates** the property (failing cases)

- **Minimising** means finding a **small / reduced failing case**
  - The minimisation is embedded into some of the tools. That is often done by operating on the test input (e.g., dividing it to obtain a smaller value, creating smaller lists, etc.)
- Minimal cases are **easier to understand** by humans and can help revealing the fault as all failures will be connected to the same violated property.

© Francisco Oliveira Neto - 2022

Claessen&Hughes, “QuickCheck: A lightweight tool for random testing of Haskell programs”, 2011.

QuickCheck

- PBT became very popular due to **QuickCheck**.
- Several PBT tools were created for **other languages**:
  - Python: Hypothesis
  - Java: jqwik
  - Javascript: fast-check

QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs

Koen Claessen
Chalmers University of Technology
koen@cs.chalmers.se

John Hughes
Chalmers University of Technology
rjmh@cs.chalmers.se

ABSTRACT

QuickCheck is a tool which aids the Haskell programmer in formulating and testing properties of programs. Properties are described as Haskell functions, and can be automatically tested on random input, but it is also possible to define custom test data generators. We present a number of case studies, in which the tool was successfully used, and also point out some pitfalls to avoid. Random testing is especially suitable for functional programs because properties can be stated at a fine grain. When a function is built from separately tested components, then random testing suffices to obtain good coverage of the definition under test.

1. INTRODUCTION

Testing is by far the most commonly used approach to ensuring software quality. It is also very labour intensive, accounting for up to 50% of the cost of software development. Despite anecdotal evidence that functional programs require somewhat less testing ('Once it type-checks, it usually works'), in practice it is still a major part of functional program development.

monad are hard to test), and so testing can be done at a fine grain.

A testing tool must be able to determine whether a test is passed or failed; the human tester must supply an automatically checkable criterion of doing so. We have chosen to use formal specifications for this purpose. We have designed a simple domain-specific language of *testable specifications* which the tester uses to define expected properties of the functions under test. QuickCheck then checks that the properties hold in a large number of cases. The specification language is embedded in Haskell using the class system. Properties are normally written in the same module as the functions they test, where they serve also as checkable documentation of the behaviour of the code.

A testing tool must also be able to generate test cases automatically. We have chosen the simplest method, random testing [11], which competes surprisingly favourably with systematic methods in practice. However, it is meaningless to talk about random testing without discussing the distribution of test data. Random testing is most effective when the distribution of test data follows that of actual data, but when testing reuseable code units as opposed to whole sys-

Source: https://blog.johanneslink.net/2018/07/16/patterns-to-find-properties/

Property-based Testing Patterns

- **Business** rules: These properties are related to the **specification**. Therefore, they cover the SUT functionalities and will be **domain-specific**.

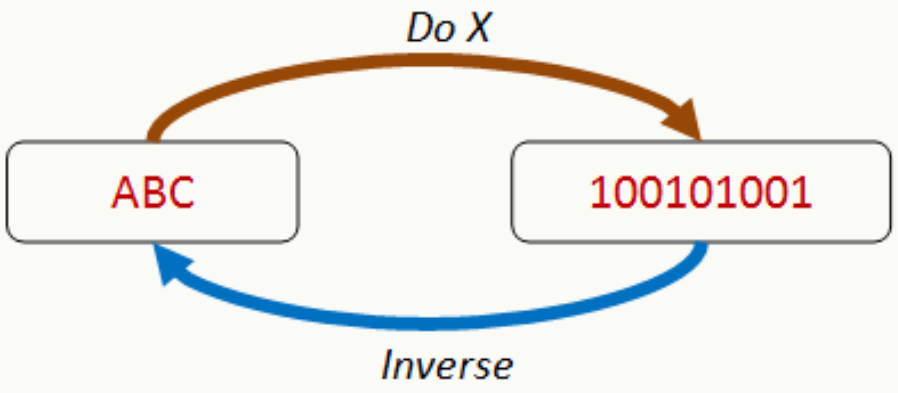
- **Fuzzying**: Properties that check whether the program crashes / breaks when providing **invalid or unforeseen input**.
  - Ex: Providing different data types as input, triggering unexpected exceptions.

© Francisco Oliveira Neto - 2022

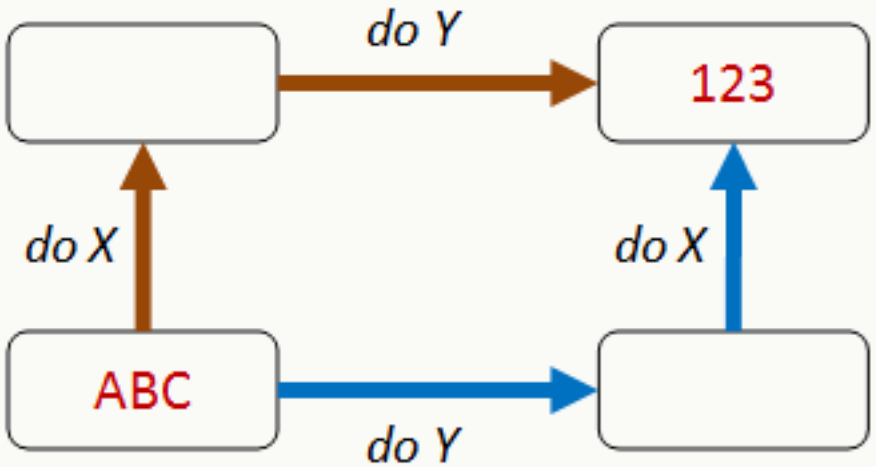
Source: https://blog.ssanj.net/posts/2016-06-26-property-based-testing-patterns.html

Property-based Testing Patterns

- Symmetry or **Inverse**: Applying a function to a value and then applying the inverse operation should **yield the original value**.
  - Ex: Parse XML to JSON and then back.



- **Commutative**: Applying the operations in **different orders** should lead to the **same result**.
  - Ex: Multiply to constant, Add matrices

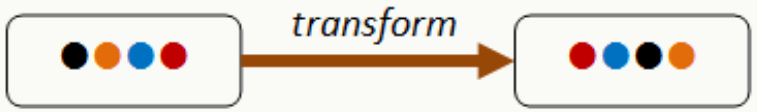


© Francisco Oliveira Neto - 2022

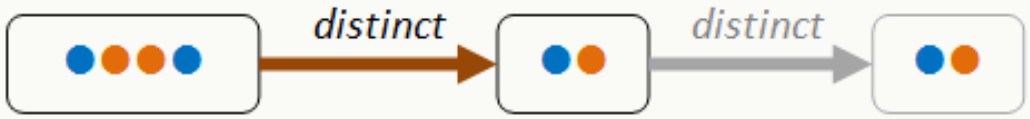
Source: https://blog.ssanj.net/posts/2016-06-26-property-based-testing-patterns.html

Property-based Testing Patterns

- **Invariants**: Identifying properties that **do not change** after specific operations.
  - Ex: Adding repeated elements to a set should not change the set size.



- **Idempotence**: Performing an **operation once** should be the same as performing an **operation multiple times**.
  - Ex: After sorting once, sorting again should yield the same result.



© Francisco Oliveira Neto - 2022

PBT – Exercise 3 – List of Strings

- A function that find names of friends in common. Receives two **sets** of strings (A and B) and returns a set with the intersection between those two lists. Throw Exception if any null item. Example:
  - `friendsA = {“Francisco”, “Beyonce”, “Phoebe”}`
  - `friendsB = {“Monica”, “Rachel”, “Phoebe”, “Francisco”}`
  - `commonFriends(friendsA, friendsB) = {”Francisco”, ”Phoebe”}`
- Write a few properties for the SUT.

© Francisco Oliveira Neto - 2022

Summary – Part 2

- PBT tools should allow (1) creation of **properties** (for all, at least, etc.), and (2) automatic **generation of arbitrary** input data.

- The generator should be **configurable** to allow custom domain needs (e.g., strings with specific format, ranges in arrays or ints)

- There are **basic patterns** when thinking of properties such as looking for commutative or idempotent aspects of the SUT.

© Francisco Oliveira Neto - 2022

Part 3:
What are the trade-off with Property-based Testing?

© Francisco Oliveira Neto - 2022

PBT - Advantages

- Tests are **easy to parameterise**, meaning that we can actually run thousands of test cases and check the same property with a **variety of test data**.
- Generating random test data is **quick** and does not require complex tools. Therefore, the **generation is cheap** and can quickly cover many cases.
- **Minimising** the failing cases **help debugging** as we can relate the failing input with the property that failed.

© Francisco Oliveira Neto - 2022

PBT - Limitations

- Properties are **harder** to define **at higher levels** of testing, therefore they are mainly used at unit-level.
  - Higher levels of testing rely on sequences of inputs (e.g., clicking buttons in an GUI), specific interactions or interdependencies (e.g., states of a database).
- **Complex input data** that have constraints are harder to generate.
  - Examples: Generating balanced trees, or XML files to be parsed to JSON.

© Francisco Oliveira Neto - 2022